

Budget-Bounded Self-Training

*a laptop-plus-under-\$1000 build plan for a Strategy-category Pokémon
TCG agent — where the deliverable is the insight, not the win rate*

Six build phases, each with tasks to hand straight to Claude, commands to run and tests to pass, an explicit compute cost, and the strategic insight it manufactures for the competition writeup. Grounded in a second, budget-filtered deep-research pass: 21 primary sources, 24 verified findings, one refutation that reshapes the design.

Deep-research run of 2026-07-12 · reframed: interpretability lives in the report, not the policy

[Companion to the decision guide ml-training-system.pdf](#)

1 Executive summary

You already have a strong agent: hundreds of hand-weighted Hypothesis rules, a turn planner, and an exact lethal solver. The problem is that improving it means another manual blunder-correction round for every deck, matchup, and rotation. This plan replaces that manual loop with a self-training system — but, because the Strategy Category judges **the reasoning and its communication**, it optimises for a different target than raw strength: an engine that manufactures communicable, evidence-backed strategic insight as a byproduct of training.

The single most important consequence of that reframe, and the one refuted claim from the research, together set the architecture:

KEY IDEA
Distilling a strong policy into a readable model **can cost skill** — the claim that it is free was refuted three votes to zero. So the interpretable layer is a reporting and translation surface over the strong policy, not a mandatory replacement for it. You run two tracks: a **value core** that learns what wins, and a **translation surface** (your named features, plus attribution and distillation) that turns what it learned into plain-English claims for the report. They meet at an **evaluation spine** that measures everything credibly under heavy luck.

The budget is not the binding constraint once you accept the reframe. The strength-ceiling option (DouZero-style self-play from scratch, \$500–960 of GPU) is **optional**. The insight-generating core — value core, automated blunder annotator, and the AIVAT evaluation spine — costs well under \$100 and runs mostly on your laptop’s CPU. Ranked by strategic insight per dollar, the build order is:

#	Phase	Cost	Primary writeup payload
1	Decision corpus + value core	\$10–30	What predicts winning, quantified
2	AIVAT evaluation spine	\$0	The rigorous methodology chapter
3	Automated blunder/insight annotator	\$0–10	Discovered blunder classes, with evidence
4	Expert iteration + feature discovery	\$0–20	New (or confirmed) strategic concepts

5	Exploiter probe	\$20–80	Robustness story: strong ≠ unexploitable
6	Optional: DMC strength ceiling	\$300–700	The interpretable-vs-neural gap

Phases 1–5 are the program. Phase 6 buys raw strength if budget and time remain, and even then its job is to **measure the gap** the report discusses, not to become the submission. If you do only three things, do 1–3: that is the entire writeup spine for under fifty dollars.

2 The ideas in plain English

New to machine learning? Read this section first — every later section leans on it. The plan uses a dozen ML terms as if they were common words. They are not, so here is each one, defined before it is used, in the language of the systems you already build. Nothing here is deep; each idea is something you could implement as a loop over an array once the jargon is stripped away.

2.1 The core trick: a model that predicts winning

You already know the shape of your agent: each situation is described by a list of numbers (**features** — “am I ahead on prizes?”, “is my active in lethal range?”), each feature has a **weight** saying how much it matters, and the **score** of a move is the weighted sum. That is a linear model, and it is 90% of the ML in this plan. Everything below is about **learning the weights automatically** and **adding a couple of new tools around them**.

Value model / value net / P(win). A function that reads the board and outputs a single number: the probability you will eventually win from here, between 0 and 1. Think of it as a **fuel gauge for the game** — not “how much damage did I just do” but “how likely am I to win overall”. Your repo already has a small one (`value_model.json`). The whole plan is built around making it good, because once you can score any position, everything else follows.

Training / regression. Teaching the model its weights by showing it examples. You feed it thousands of past positions, each labelled with who actually won, and an algorithm nudges the weights over and over to make its predictions match the outcomes — the same idea as least-squares curve-fitting, just with more inputs. “Regression” is just the word for fitting a number-predicting function to data.

Calibration. Whether the fuel gauge is honest. If the model says “70% to win” on a thousand positions, you should win about 700 of them. A **well-calibrated** model’s numbers mean what they say. **Brier score** and **log-loss** are just two ways to score calibration — lower is better — the way you would check a sensor against ground truth before trusting its reading.

Self-play. The agent plays against copies of itself, over and over, to generate an endless supply of labelled games **for free**. This is your data source: no humans, no downloads, just your simulator running overnight. Your 1000 games/minute is the whole fuel supply.

Deep Monte-Carlo. Two words. **Monte-Carlo** means “estimate something by trying it many times and averaging” — like estimating a dice’s fairness by rolling it a thousand times instead of doing the probability algebra. **Deep** means “use a neural network to store those averages”. Put together: play millions of self-play games, and for each position, learn the average outcome. That is how the value model gets trained. It is **cheap per game but needs a lot of games** — the opposite of clever, but it parallelises perfectly, which is why it fits a laptop plus a cheap GPU rental.

Neural network. A flexible function with many tunable numbers that can approximate complicated input-output relationships a straight-line model cannot. You only need one here if the simple linear model fails its calibration check — reach for it last, not first.

Discount (γ). A knob for “how much do I value a reward later versus now”. Setting it to 1.0 means a win counts fully no matter which turn it arrives — correct for us, because the only

reward is winning the game, and a win on turn 30 is worth exactly as much as a win on turn 8.

2.2 The two hard problems: luck, and blame

Card games have two properties that break naive approaches, and two named tools fix them.

Credit assignment. The blame problem. A match is a long chain of decisions with a single win-or-lose at the very end. If you lose, **which** decision was the mistake? Naively blaming every move in a loss (and praising every move in a win) is what your manual corrections quietly avoid, and it is wrong: you can play perfectly and lose to bad shuffles. The fix uses the value model: walk the game, and at each decision ask “did my win-probability **drop** when I chose this move instead of the best available one?” That drop, written $\Delta P(\text{win})$, is the automated blunder detector — exactly what a chess engine does when it annotates a game with “?! ” and “?? ” marks.

Variance, and variance reduction. The luck problem. Card games are so swingy that if agent A is genuinely 2% better than agent B, you might have to play **tens of thousands** of games before the win-rate reliably shows it — the shuffles drown the skill. **Variance reduction** is a statistical trick that subtracts out the luck you can account for (you know the deck’s odds, you know your own strategy) and leaves the skill signal — like noise-cancelling headphones for randomness. It does not change the true answer, it just lets you hear it with far fewer games.

AIVAT. The specific, proven variance-reduction method this plan uses. Its guarantee is that it is **unbiased** — it never distorts the real answer, it only sharpens it — even if the value model you plug into it is mediocre. In poker it cut the number of games needed for a confident conclusion by more than 10×. This is the fix for your “cross-deck gauntlet was uninformative” problem: the gauntlet was not lying, it was just too noisy to hear.

Paired seeds / duplicate deals. The simplest luck-canceller. Give both agents being compared the **exact same shuffles**, and mirror who goes first — so if a deal was lucky, both sides get that luck and it cancels. This is literally how duplicate bridge tournaments remove the deal from the scoring.

2.3 Teaching a fast policy: teacher and student

The runtime agent must decide in seconds on a CPU. These tools let a slow, strong “teacher” pour its skill into your fast rule layer.

Expert iteration (teacher / student). The teacher — here, your planner plus the value model, allowed to think hard — is strong but too slow to ship. The **apprentice** — your fast linear rule layer — watches the teacher’s choices and adjusts its weights to imitate them, absorbing the teacher’s skill into something that runs instantly. You keep your interpretable rules; they just get **taught** instead of hand-tuned.

DAgger. A refinement of the above. Instead of only showing the student the teacher’s games, you let the **student** drive and have the teacher correct it at every step — so the student also learns how to recover from its own mistakes, not just how to copy a perfect game. (The name stands for Dataset Aggregation and does not matter; the let-the-student-drive idea does.)

Behavior cloning. The bluntest copy: train a model to predict “what move did that agent make in this state?” from a log of its games. Pure imitation, no notion of winning — but a surprisingly strong starting point, and dirt cheap. It is the warm-start for the exploiter below.

Reinforcement learning / PPO. Learning by trial and error: the agent tries moves, gets rewarded for winning, and shifts toward what worked. **PPO** is just the most popular, stable recipe for doing this without the training blowing up — you use it off-the-shelf from a library, you do not implement it.

Best response / exploiter. Train a fresh opponent whose **only** goal is to beat **your** agent, and see how badly it can. If a cheaply-trained exploiter crushes you, your agent has a blind spot worth knowing about **before** the competition finds it. A strong agent is not automatically a robust one — this is how you check.

2.4 Reading what the machine learned

The point of the whole exercise is a report full of communicable strategy, so these tools turn a trained model back into words.

Feature construction. Instead of hand-writing every rule, let the system **notice** that two conditions occurring together predict a mistake, and mint that combination as a new feature automatically. This is the machine proposing its own strategic hypotheses — the thing that stops you hand-authoring rules forever.

Distillation. Take a strong but hard-to-read model and train a simple, readable model (a small decision tree, say) to imitate it — so you can **read** the strategy the strong one found. The catch, proven in the research and central to this plan: the readable copy can be **weaker** than the original. So you keep the strong model as the one that plays, and use the readable copy only to **explain** it in the writeup.

Set encoder / permutation-invariance. A hand of cards has no meaningful order — “Pikachu then Oran Berry” is the same hand as “Oran Berry then Pikachu”. A naive model wastes effort relearning what a Pikachu is in every hand slot. A **set encoder** treats the hand as an unordered bag, so each card is learned once. You only need this if the simple model underperforms — it is a data-efficiency upgrade, not a starting requirement.

Translation surface. Not a standard term — it is how this plan uses your named-feature layer. Because every feature has a plain-English name and a weight, the model’s knowledge reads as sentences (“being ahead on prizes is worth three times avoiding a knockout”). That readability is what converts training results into report content — the feature layer **translates** what the machine learned into what you write down.

KEY IDEA

If you remember only one thing: the value model — the win-probability fuel gauge — is the keystone. It is the blunder detector (credit assignment), the luck-canceller’s helper (AIVAT), and the teacher’s brain (expert iteration), all at once. Build that one small thing well and the rest of the plan is wiring around it.

3 What the reframe changes

The first report assumed the runtime policy had to stay interpretable, and built the whole architecture around keeping the rule layer as the deployed policy. The competition description dissolves that constraint: **“the Strategy Category evaluates the participant’s strategic logic... Why was a particular strategy chosen? What hypotheses were tested? ... middle or lower tiers can still achieve high overall scores through deep analysis, originality, and well-structured reporting.”** Interpretability is a property of the report and the research process, not of the agent’s decision mechanism.

Three things change, concretely:

The strength ceiling comes off the policy. You may deploy whatever plays best — a neural value net, deeper search — because the writeup, not the policy, carries the interpretability load.

The experimentation apparatus becomes the product. What earns Strategy points is the trail of tested hypotheses and measured effects. So the annotator, the ablation harness, the AIVAT protocol, and the feature-discovery loop stop being supporting infrastructure and become the things that **generate the deliverable**. A system that emits tested strategic claims is worth more than a black box that merely wins, because the claims are literally what is graded.

The hybrid survives, re-justified. Not “the policy must be glass-box,” but “we want a substrate that emits readable strategic claims.” A learned discovery expressed in the vocabulary of named Hypotheses is a report sentence with evidence attached; the same discovery buried in neural weights has to be reverse-engineered. The named-feature layer earns its place as the **translation surface**.

FOR THIS REPO

You are unusually well-positioned for this. Your value model (`src/common/value/value_model.json`) is **already** a linear model over roughly seventeen named strategic features — `prize_diff`, `favorability`, `active_doomed`, `line_ready`, `incoming_active_damage`, and so on. It is not a black box that needs distilling; it is **born** as a translation surface. Every weight in it is a sentence: “being ahead on prizes is worth w times as much as having your active in lethal range.” Phase 1 mostly makes that model richer and better-calibrated, not fundamentally different in kind.

The one hard lesson from the research that constrains this: **do not assume the readable model is free**. If a distilled decision tree or a linear surrogate loses strength versus the model it explains, keep the strong model as the runtime and use the readable one only to **narrate** it. The report can honestly say “our deployed policy scores X ; the human-readable distillation that explains it scores $X-\epsilon$, and here is what the gap tells us.”

4 The budget, in real numbers

The research pinned down where the compute walls actually are, which is what lets us reject the expensive paths cleanly and size the cheap ones.

Reference system	What it cost	Verdict	What we borrow
Suphx (Mahjong, top 0.01% human)	44 GPUs, 2 days, 2.5M games	Far over budget	The method only: value-delta credit assignment ($\Delta\Phi$), at tiny scale
DouZero (DouDizhu, SOTA)	4 GPUs, days, \$500–960	The affordable ceiling	Deep Monte-Carlo recipe if Phase 6 is funded; card-matrix action encoding
Showdown PPO (>1900 ELO)	1× RTX 3090, 2 days, 250M states	Consumer-hardware proof	BC warm-start → self-play; set-style token encoder; distributional value head
ByteRL exploiter probe	1 BC + 1 PPO best-response	Cheap, in budget	The whole robustness diagnostic (Phase 5)
AIVAT evaluation	CPU only, no training	Effectively free	The entire evaluation spine (Phase 2)
Coac (LOCM, 94% win)	depth-3 alpha-beta, no learning	Free; beat RL for years	Validation that your planner is the right expert to warm-start from

Two research nuances matter for sizing. First, Deep Monte-Carlo is **compute-cheap but sample-hungry** — its leverage is parallel throughput (billions of cheap games), not few samples; this is fine for you because your simulator already does 1,000 games/min and generates data on the CPU for free. Second, and decisively: cheap adversarial search beat learned policies at LOCM for years (Coac’s depth-3 alpha-beta won multiple titles at up to 94% before neural nets caught up only as complexity rose). That is the empirical licence to **warm-start from your existing planner + lethal solver as the expert** rather than train a policy from scratch — the single biggest cost saving in this plan.

FOR THIS REPO

Your marginal cost is dominated by **rented GPU hours for training the value core and one exploiter PPO run**. Everything else — self-play data generation, AIVAT evaluation, the annotator, feature discovery, expert-iteration weight fitting — is CPU work that runs on your laptop or on the simulator you already have. Rent a single modern GPU on a spot instance (order \$0.30–1.00/hour) only for the neural-training bursts, an hour or

few at a time. The insight-core program (Phases 1–5) realistically lands between \$50 and \$200 total.

5 The architecture: three subsystems

Everything below is one of three subsystems, wired to pieces you already have.

The value core — a calibrated win-probability model $P(\text{win})$ conditioned on the board state. Trained by regressing self-play outcomes (Deep Monte-Carlo’s core idea, minus the expensive policy extraction). It is the engine behind credit assignment, the annotator, and the AIVAT estimator. Seeded by the existing linear `value_model.json`; kept as a named-feature model as far as calibration allows, so it stays a translation surface.

The translation surface — the named Hypothesis features, plus attribution (which features moved a decision), plus optional distillation. This is what converts the value core’s knowledge and the annotator’s findings into report sentences. Anchored on the Soemers/Ludii result: interpretable linear pattern-features, weights learned and **feature set grown automatically during self-play**, strong after few games, and readable as plain-English strategies.

The evaluation spine — AIVAT variance reduction over paired-seed, mirrored, skill-stratified deals. This is what makes every claim in the report **credible**, and it directly fixes the gauntlet-was-uninformative problem. It reuses the value core as its heuristic estimator and `cgpy`’s deterministic seeds + perfect-information tap as its scaffolding.

KEY IDEA

The subsystems share artifacts, which is why the program is cheap: the **same** value core is the credit signal (Phase 3), the AIVAT estimator (Phase 2), and the expert’s evaluator (Phase 4). You train one small model well and spend it five ways.

Repo pieces this builds on, verified present on this branch unless noted: `tools/sim/battle.py` (self-play / A-B harness), `tools/train/tuner/` (the weight-fitting pipeline — `featurize.py`, `fit.py`, `propose.py`, `attribution.py`, `retest.py`, `run.py`), `data/corrections/reviewed.json` (the human blunder ledger — your calibration set), `src/common/value/value_model.json` (the linear value model), `src/common/scouting/` (the Read + Briefs), and `live_trace` telemetry. The pure-Python engine twin `src/cgpy/` (ADR-0050 — deterministic seeds, perfect-information tap) lives on its own branch and must be merged in before Phase 2.

6 The build plan

Each phase gives a one-line goal, a **Build** block written to hand to Claude, a **Run & test** block of commands and acceptance criteria for you, the compute cost, and the **Writeup insight** it produces. Phases are ordered by insight-per-dollar; each is independently valuable and kill-switched.

6.1 Phase 1 — Decision corpus and the value core

Goal: log every decision, and turn the existing linear value model into a calibrated $P(\text{win})$ engine trained across all deck pairings.

IN PLAIN ENGLISH

You are building the **win-probability fuel gauge**. You save every decision your agent faces plus who eventually won that game, then fit a model to predict “chance of winning from here”. **Calibrated** means honest: when it says 70%, it wins about 70% of the time. You keep it a weighted sum of named features (not a black box) so it stays readable.

BUILD — FEED TO CLAUDE

1. Extend `live_trace` to emit one structured record per decision: full engine observation, the legal option menu as structured actions, the active Hypothesis feature activations, the Read’s archetype posterior, the planner’s verdict, the chosen action, and (filled in at match end) the terminal win/loss. Write to a JSONL corpus keyed by match seed + ply.
2. Add a corpus-generation mode to `tools/sim/battle.py` that runs self-play across **all** deck pairings (not just mirrors) at full throughput and appends decision records.
3. Extend the value-model training (the `tools/train/tuner/` pipeline, `featurize.py` + `fit.py`) to fit $P(\text{win})$ conditioned on state over this corpus. Keep it a linear/GAM model over the **named** features in `value_model.json`, extended with any decision-context features already computed by the Pilot. Only add a small permutation-invariant set-encoder head (Deep Sets style over bench/hand card tokens) **if** the calibration gate below fails on the linear model.
4. Train with discount $\gamma = 1.0$ (no in-match discounting — win/loss is the only reward) and mask illegal actions structurally.

RUN & TEST — YOU

- Generate a day of self-play on the laptop/CPU (free): `python tools/sim/battle.py --corpus ...`. Confirm the JSONL loads in a notebook and one match round-trips (records reconstruct the game).
- Train the value head on a rented GPU for an hour or few (`fit.py`); **cost** $\approx$ \$10–30.
- **Gate:** on held-out matches from deck pairs unseen in training, the model’s calibration (reliability curve, Brier score, log-loss) must beat both the current `value_model.json`

and a plain logistic baseline. If the linear model cannot calibrate, add the set-encoder head and retrain; re-gate.

WRITEUP INSIGHT

“Here is what predicts winning in this format, quantified.” The fitted feature weights are a ranked, evidence-backed statement of what matters — prize tempo versus board development versus lethal threat — and the calibration curve is proof the model **understands** the game, not just memorises it. This is the report’s opening empirical claim and the foundation every later insight cites.

6.2 Phase 2 — The AIVAT evaluation spine

Goal: replace uninformative gauntlet winrates with a provably-unbiased, variance-reduced, paired-seed, skill-stratified measurement protocol.

IN PLAIN ENGLISH

This phase is **noise-cancelling headphones for luck**. Raw win-rate is so drowned in shuffle luck that a real 2% edge can hide for tens of thousands of games. Three fixes stack: give both agents the **same shuffles** so luck cancels (paired seeds); use AIVAT to mathematically subtract the luck you can account for (provably without distorting the true answer); and report separately on the deals where skill **could** matter (stratification). Result: you can trust a conclusion from 10× fewer games — which is why your old gauntlet felt useless.

BUILD — FEED TO CLAUDE

1. Build a paired-seed / mirrored-deal harness on cgpy’s deterministic seeds: sample a fixed set of shuffles, play each configuration on the **same** seed for both agents, and mirror every match for first/second player (the LOCM protocol). This alone removes most deal variance.
2. Implement AIVAT correction terms over the TCG game tree: enumerate the chance events (opening hand, draws, prize flips, any coin flips) with their known probabilities, and the known-strategy correction for our own agent whose policy is explicit. Use the Phase-1 value core as the required heuristic state-value estimate — AIVAT is provably unbiased **regardless of that estimate’s quality**, so an imperfect value net is safe.
3. Implement skill-sensitivity stratification: for each seed, run a perfect-information “cheating” agent (via cgpy’s perfect-info tap) against the baseline; label seeds where cheating wins big as skill-sensitive. Report effects on that stratum separately.

RUN & TEST — YOU

- **Validation of unbiasedness:** at large N, the AIVAT estimate must converge to the raw winrate mean (they estimate the same quantity). Assert this in a test.

- **Validation of power:** take a known-difference pair — lethal solver on versus off — and show the AIVAT + paired-seed estimate reaches significance with **an order of magnitude fewer games** than raw winrate. Reproduce the “>10× fewer samples” figure on your own game.
- **Cost $\approx$ \$0** — pure CPU.

WRITEUP INSIGHT

This **is** a report chapter, and an original one: “Naive cross-deck gauntlet winrates were uninformative — they failed to separate agents we knew differed. We built a variance-reduced evaluation protocol (AIVAT + paired seeds + skill-sensitivity stratification) that detects a two-percent edge with ten times fewer games, and here is the math and the before/after confidence intervals.” Rigorous experiment design is exactly what the Strategy Category rewards, and almost no competitor will have it.

6.3 Phase 3 — The automated blunder / insight annotator

Goal: per-decision $\Delta P(\text{win})$ flags systematic misplays automatically, calibrated against your months of human corrections; clustered flags become named blunder classes.

BUILD — FEED TO CLAUDE

1. Implement a chess-engine-style annotator that walks every logged match, scores all legal options at each decision under the value core (add a shallow planner rollout where it is cheap to confirm), and computes the win-probability drop $\Delta = P_{\text{best}} - P_{\text{taken}}$ — the value of the best available action minus the value of the action actually taken.
2. Emit decisions below a threshold as Correction records into the existing pipeline, with an automated-provenance tag, so the human review surface sees **flags**, not raw replays.
3. Calibrate the threshold against `data/corrections/reviewed.json`: sweep it to trade precision against recall of human-confirmed blunders.
4. Cluster the flags by Hypothesis-activation signature and board context (e.g. archetype, prize count, active-doomed) to name recurring blunder **classes** rather than one-off mistakes.

RUN & TEST — YOU

- **Gate:** on a held-out slice of `reviewed.json`, the annotator recovers a majority of human-confirmed blunders at a false-flag rate a weekly review can absorb; report precision and recall against the human ledger.
- Spot-check the top clusters by hand — each named class should correspond to a real, describable mistake.
- **Cost $\approx$ \$0–10** (inference over logged games).

WRITEUP INSIGHT

Two payloads. First, the hypotheses-tested content: “The system automatically discovered these N systematic blunder classes — e.g. **over-attaching to a doomed active, developing behind tempo when a KO is available** — each with $\Delta P(\text{win})$ evidence and frequency.” Second, a validation-of-method story: “Our automated labeler agrees with months of human review at precision/recall X/Y ,” which justifies retiring the manual loop. This is the heart of “what hypotheses were tested.”

6.4 Phase 4 — Expert iteration and automatic feature discovery

Goal: tune weights **and grow the feature set automatically**, warm-started from the planner as the expert; harvest new (or confirmed) strategic concepts for the writeup.

IN PLAIN ENGLISH

This is **teacher and student**. The teacher (planner + value model, thinking hard) is too slow to ship; the student (your fast rule layer) watches the teacher’s choices and adjusts its weights to copy them — so your interpretable rules get **taught** instead of hand-tuned. The bonus: the system also **proposes new rules** by noticing which pairs of conditions, occurring together, predict a mistake. One guardrail from the research — the student must learn against a **competent** opponent, never a random one, or it learns to exploit nonsense.

BUILD — FEED TO CLAUDE

1. Set up expert iteration: the **expert** is the planner + value core under shallow search; the **apprentice** is the linear Hypothesis layer, which remains the runtime policy. The per-decision training target is the expert’s choice distribution; fit via the existing tools/train/tuner/ pipeline, extending `fit.py` from the perceptron update toward cross-entropy SGD against the expert target (a DAgger-style loop — query the expert at visited states).
2. Add Soemers-style automatic feature construction: mine co-active Hypothesis pairs whose conjunction correlates with apprentice-versus-expert error, mint the conjunction as a candidate feature, and let the tuner weight it (weights may be negative, to discourage a pattern). Redundancy-penalise against constituents; prune to the top-magnitude weights and **measure the per-decision feature-evaluation cost** against the CPU/time budget.
3. Critical guardrail from the research: evaluate candidate features against the **competent** planner opponent, never random self-play. Frequent-pattern mining from random games was shown to fail outright, and training against a random opponent yields degenerate exploits.
4. Route matchup-conditioned weights through the existing `Strategy.weight_overrides` carrier — learned per archetype rather than hand-authored.

RUN & TEST — YOU

- **Gate:** the Phase-2 protocol shows non-regression versus the hand-tuned baseline across the stratified deal pool, **and** the Phase-3 annotator's flags-per-thousand-decisions falls.
- Hand-inspect every newly minted feature for meaning; keep only the ones that read as a strategy.
- Learned weight changes land as reviewable diffs against named Hypotheses; revert is a config flip.
- **Cost** $\approx$ \$0–20 (mostly CPU self-play; a little GPU if the value core is refreshed).

WRITEUP INSIGHT

The originality payload — with an honest fork. Best case: “the system discovered a strategic concept we had not hand-authored” (for instance, a context where a feature should split three-to-one by matchup). Expected case, still valuable: it **re-derives** concepts you already encoded — which is itself a validation story (“an independent automatic process recovered our hand-authored rules, confirming they capture what wins”). Either way you can show **how the design reflects understanding of the game's mechanics**, with the mechanism on display.

6.5 Phase 5 — The exploiter probe

Goal: measure how exploitable your strong agent is, cheaply, and feed the discovered weaknesses back — the ByteRL recipe as a diagnostic.

IN PLAIN ENGLISH

You hire a **sparring partner whose only job is to beat you**. First cheaply copy your own agent (behavior cloning — imitate its moves from logs). Then let that copy learn, by trial and error (reinforcement learning, using the off-the-shelf PPO recipe), with one goal: exploit your agent. If it finds an easy win, you have a blind spot — fix it now, before the competition's opponents find it. Strong does not mean unbeatable, and this is how you check the difference.

BUILD — FEED TO CLAUDE

1. Behavior-clone the current agent from its own self-play logs (a supervised classifier of its action given state). Expect the clone alone to reach roughly parity-minus — around 40% against the real agent — which is your warm start.
2. Freeze the agent into the environment (making it part of the world) so the exploiter is a **single-agent** RL problem — the tractable, low-compute framing — and PPO-fine-tune the clone as a best response. Use a laptop-friendly library (Stable-Baselines3 or CleanRL).

3. Report the exploiter's win rate against the frozen agent as a standing dashboard number. When the exploiter finds a repeatable line, encode the counter as a new Brief or correction and re-run.

RUN & TEST — YOU

- **Gate:** exploiter win rate below an agreed ceiling. Before/after a weakness fix, show the exploiter's edge drops.
- **Sanity:** the frozen-opponent env is deterministic given seed; the PPO run should be reproducible.
- **Cost $\approx$ \$20-80** — one behavior-clone (CPU) plus one PPO best-response run (a GPU-day at most; the Showdown-PPO reference of a 3090 for two days is the upper bound, and yours is warm-started and smaller).

WRITEUP INSIGHT

A robustness-and-honesty narrative that judges reward: "Our agent is strong, but strong does not mean unexploitable — a best-response opponent trained in a day reached W% against it by exploiting Z. We hardened the line and W dropped to W'." The ByteRL precedent (beat a top-10 human, yet 90% exploitable in LOCM) is your citation that this failure mode is real and worth probing. This is **strength across matchups and the ability to keep winning**, examined honestly.

6.6 Phase 6 — Optional: the Deep Monte-Carlo strength ceiling

Goal: if budget and time remain, push raw strength and meta-robustness — and **measure the gap** between the interpretable hybrid and the neural ceiling for the report.

BUILD — FEED TO CLAUDE

1. Train a DMC value/policy by self-play (the DouZero recipe): card-matrix action encoding, parallel actors, sampled-return targets, no tree search. Warm-start with oracle guiding — feed the perfect-information features from cgpy's tap and anneal them to zero on a decaying dropout schedule, so the deployed model uses only public information.
2. For the meta, run a **small** SP-PSRO population — it converges in a few iterations in two-player zero-sum, so a handful of policies approximates the equilibrium; avoid full double-oracle blowup.
3. If you deploy a neural policy, distill it back into the feature vocabulary (VIPER-style, Q-weighted DAgger) **for the report only** — remember distillation-is-free was refuted, so keep the neural policy as runtime if it is stronger and use the tree purely to narrate it.
4. Consider pMCPA-style per-match adaptation **only** if the 10-minute CPU grader budget allows the extra rollouts (Suphx's adapted agent beat its static self 66% of the time with few sims).

RUN & TEST — YOU

- Evaluate with the Phase-2 AIVAT protocol against the Phase-4 hybrid; the Kaggle ladder is the final arbiter.
- **Gate:** the neural policy must beat the hybrid on the stratified, variance-reduced protocol by a margin the CIs actually support — not a raw-winrate mirage.
- **Cost $\approx$ \$300–700** — the DouZero-scale item, and the reason this phase is optional.

WRITEUP INSIGHT

“We measured the ceiling. A from-self-play neural policy reaches strength S ; our interpretable hybrid reaches $S-\delta$; and distilling the neural policy back into named features shows the extra δ comes from behaviour B ” — the honest quantification of what interpretability costs, which is a strong, original discussion section precisely because so few entrants will have both halves to compare.

7 Turning the build into the writeup

The competition asks four questions. Map the phases straight onto them; the report almost writes itself because each section is an experiment you already ran.

Competition question	Answered by
Why was a particular strategy chosen?	Phase 1 value-core weights (what wins) + Phase 4 expert-iteration (why these weights, learned not guessed)
What hypotheses were tested?	Phase 3 discovered blunder classes + Phase 4 minted/confirmed features — each a hypothesis with $\Delta P(\text{win})$ evidence
How does the design reflect understanding of the game’s mechanics?	Phase 1 named-feature model + Phase 2 chance-event decomposition (you had to model draws, prizes, flips to build AIVAT)
Strength across matchups / ability to keep winning?	Phase 5 exploiter probe + Phase 2 per-matchup stratified results

The exemplars that scored well share a shape worth copying. The strongest game-AI competition writeups (DouZero’s paper, the LOCM organiser reports, the Coac summary) lead with the **problem structure** — why the naive approach fails (here: manual tuning does not scale; gauntlet winrates are uninformative) — then present **a method chosen for a stated reason under a stated constraint**, then **quantified experiments**, then an **honest limitations** section. The reframe means your report is not “we built a strong bot”; it is “we built a machine for discovering and validating strategy, and here is what it found.” Structure it as: the scaling problem → the evaluation problem and its fix (Phase 2) → automated discovery of what wins and what loses (Phases 1, 3) → learning and growing the strategy (Phase 4) → stress-testing it (Phase 5) → the interpretability-versus-strength frontier (Phase 6, if run) → limitations and what you would do next.

KEY IDEA

Write the report **as you build**, not after. Every gate you pass is a figure; every threshold sweep is a table; every discovered blunder class is a paragraph. The AIVAT before/after confidence intervals, the annotator’s precision/recall against the human ledger, and the exploiter’s win-rate drop after hardening are three publication-quality results that cost almost nothing beyond the build itself.

8 Cost ledger, risks, and kill criteria

8.1 Where the money goes

Item	Est. cost	Notes
Self-play data generation	\$0	Laptop/desktop CPU, the simulator you have
Value-core training (Phase 1)	\$10–30	A few GPU-hours on a spot instance
AIVAT + evaluation (Phase 2)	\$0	Pure CPU
Annotator (Phase 3)	\$0–10	Inference over logged games
Expert iteration (Phase 4)	\$0–20	CPU self-play + occasional value-core refresh
Exploiter PPO run (Phase 5)	\$20–80	One best-response, warm-started
Insight core subtotal (1–5)	\$50–140	The whole writeup spine
DMC strength ceiling (Phase 6)	\$300–700	Optional; DouZero-scale self-play
Total if you do everything	\$350–840	Comfortably under the \$1000 ceiling

8.2 Risks, and how the plan contains them

Domain transfer is the top uncertainty. Every cited result is from Mahjong, poker, DouDizhu, general board games, Atari, or LOCM — none is Pokémon TCG. The methods are sound; their exact numbers (AIVAT’s “>10×”, DouZero’s days) may not reproduce identically. Containment: the Phase-2 gate **measures** your actual variance reduction rather than assuming the poker figure, and every phase is independently useful.

AIVAT needs engineering, not just a citation. Its unbiasedness is a theorem, but you must define the chance and known-strategy correction terms over your specific game tree. This is real work — budget for it — but the cgpy engine twin’s perfect-information tap and deterministic seeds satisfy its preconditions, and the payoff (a rigorous methodology chapter) is worth it.

Feature discovery may only re-derive. The automatic feature construction might surface nothing your hundreds of hand-authored rules do not already encode. Contained by reframing: re-derivation **is** a validation result worth reporting, and the mechanism is on display either way.

Self-play bias. The value core only sees states your own policy visits, so a Pilot blind spot becomes an oracle blind spot. Mitigations already in the plan: the Phase-5 exploiter deliberately hunts those blind spots, and oracle guiding (Phase 6) injects perfect-information states.

Do not predict the opponent’s hand naively. The research carried a specific caution — an opponent-hand-prediction extension (ProphetCoac) **reduced** LOCM win rate. If you add opponent modelling, gate it behind the Phase-2 protocol before trusting it.

8.3 Kill criteria

Stop or reroute a phase when: the value core cannot beat the existing linear model on calibration (Phase 1 — the keystone is cracked; fall back to planner-rollout labels for the annotator and skip the learned value core); the annotator's flags disagree wildly with the human ledger (Phase 3 — insert planner-verified rollouts between value net and label, or narrow to lethal-adjacent decisions where the oracle is exact); or Phase 6's neural policy fails to beat the hybrid on the **variance-reduced** protocol (then it is a raw-winrate mirage — do not ship it, and the **negative** result is still a report paragraph).

9 Source annex

Second deep-research pass (2026-07-12, run wf_322af7fc-d9b): five budget-filtered angles, 21 primary sources, 88 claims extracted, top 25 adversarially verified — 24 confirmed, one refuted. The refuted claim reshapes the design and so is listed first.

Source	What it contributes	Status
Bastani et al., VIPER (NeurIPS 2018) arXiv 1805.08328	Distillation to trees — but “distilled tree matches oracle for free ” was REFUTED 0-3: readable $\neq$ free	Refuted claim
Suphx (MSR 2020) arXiv 2003.13590	Value-delta credit ($\Delta\Phi$, GRU+2FC); oracle guiding; pMCPA; entropy reg — and the 44-GPU floor to borrow small	Verified
DouZero (ICML 2021) arXiv 2106.06135	Deep Monte-Carlo; 4 GPUs/days $\approx$ the affordable ceiling; card-matrix action encoding; compute-cheap but sample-hungry	Verified
AIVAT (AAAI 2017) arXiv 1612.06915	Provably-unbiased variance reduction; plug in any heuristic value; $>10\times$ fewer games; naive MC fails to reach significance	Verified
Soemers/Ludii features (2019) arXiv 1903.08942	Linear pattern features, weights learned + set grown automatically; readable strategies; must train vs a competent opponent	Verified
SCoBots (NeurIPS 2024) arXiv 2401.05821	Concept bottlenecks as insight and diagnostic — surfaced a real Pong misalignment	Verified
Haluška & Schmid, exploiting ByteRL (2024) arXiv 2404.16689	BC $\rightarrow$ PPO best-response recipe; strong $\neq$ robust (90% exploitable); freeze-opponent single-agent framing	Verified (medium)
LOCM competition summary (2023) arXiv 2305.11814	Coac depth-3 alpha-beta won at 94% — cheap search beat RL for years $\rightarrow$ warm-start from your planner	Verified
Showdown PPO writeup	RTX 3090 $\times$ 2 days $\rightarrow$ >1900 ELO; BC warm-start; 15-token set encoder + distributional value head	Extracted
Metamon offline RL (2025) arXiv 2504.04395	Search-free offline RL from replays; multi- γ helps; full config above budget (aspirational)	Extracted
Warm-Stagger hybrid IL (2024)	Offline demos + interactive expert queries $\geq$ better of either; state-wise annotation is horizon-independent	Extracted

SP-PSRO (2022) arXiv 2207.06541	Converges in a few iterations — a small population approximates Nash for cheap meta-robustness	Extracted
poke-env / RLCard / SB3 / CleanRL	Laptop-runnable Python tooling for local self-play and the exploiter PPO run	Tooling

Provenance. Generated from the 2026-07-12 budget-filtered deep-research run (wf_322af7fc-d9b): five search angles, 103 agents, 21 sources fetched, 88 claims extracted, 25 verified by three-vote adversarial refutation (24 confirmed, 1 refuted). Machine-readable findings are archived with the run. This playbook is the implementation companion to the decision guide docs/research/ml-training-system.pdf; together they cover **why** (that document) and **how** (this one). In-repo companions: docs/tuning/methodology.md, docs/architecture/tiers.md, ADR-0042 (value model), ADR-0050 (cgpy engine twin).